Do We Train on Test Data?

The Impact of Near-Duplicates on License Plate Recognition: what the paper says, what our two Kaggle runs found, and how to use it

Prepared for MD. Rokib Hasan Oli (paper reader and data lead) · Supervisor: Dr. Md Mansur ul Mulk · 22 October 2026 (replaces the report of 18 October)
Paper: R. Laroca, V. Estevam, A. S. Britto Jr., R. Minetto, D. Menotti, IJCNN 2023, DOI 10.1109/IJCNN54540.2023.10191584, arXiv 2304.04653v2 (8 pages). Page numbers below are the arXiv PDF pages.
Main run: `paper23_kaggle.ipynb` on dataset v1.1, finished 22 Oct 2026, 96.3 min on 2× Tesla T4. First run: dataset v1.0, 18 Oct 2026 (kept for comparison in section 13).

What changed since the first report. Dataset v1.1 fixed the paper #23 reading lists. Both plate readers are now tested on **the same 194 KUET test plates**, and none of those plates is in the fair reader's training data. Both versions also train for about the same number of steps, and a paired significance test (McNemar) is added. The headline therefore changes: 85.57 % vs **77.84 %** on the same plates (error ×1.54, p = 0.008). The first report's 85.57 % vs 72.13 % (×1.93) compared different test plates and is replaced.

The whole report on one page

The paper in three lines. The two most used LPR datasets (AOLP, CCPD) put different photos of the *same plate* in both train and test. 47-68 % of AOLP test images and 19 % of CCPD test images have such a "near-duplicate" in training. When the authors re-split the data so no plate is on both sides, six OCR models' error rates more than double on AOLP, CCPD's average accuracy falls from 80.3 % to 77.6 %, and the "best" model changes (p.1-5).

Our result in three lines. We trained the same YOLOv8n plate detector and plate reader twice on the KUET Bangla dataset: once on KUET's published split with its Roboflow copies ("leaky"), once on our grouped split ("fair"). Both readers are tested on the same 194 KUET test plates. 114 of those plates (59 %) are also in the published training set. The leaky reader reads **85.57 %** of the plates and the fair reader **77.84 %**, so the error grows from 14.4 % to 22.2 % (×1.54, McNemar p = 0.008). The gap comes from the 114 plates the leaky reader had seen (+10.5 points, p = 0.017). On the 80 plates neither saw (+3.75, p = 0.45) and on 468 plates from other datasets (50.2 % vs 50.0 %) there is no real difference.

Is the training OK? Yes. All 5 jobs ended with `code=0` on dataset v1.1, with 0 corrupt images and no NaN. The two leaky models reproduce the 18 October numbers to the last digit. The fair reader stopped at epoch 377 of 404 by design (no gain for 100 epochs). There are no errors, but there are two things to know before you quote numbers (section 15): the fair detector overfits a little with the longer training, and the kit's per-condition reading table that includes KUET crops is not a fair comparison.

What to tell the faculty. "Public Bangla LPR splits leak test plates into training. On KUET, 59 % of the test plates are also in training, and this alone makes plate reading look 7.7 points better than it is (p = 0.008). On new plates the 'better' model is not better. That is why our dataset uses a grouped split." Show 5 things: the problem picture, the paper's table, our leakage audit, the same-plates result and the seen/unseen chart (section 18).

GitHub. A ready folder is in `After Train/github_upload/weights/paper23/MD.Rokib.Hasan.Oli/`; the steps are in section 22.

Organisation. Part A (sections 1-9): the paper. Part B (10-15): our experiment, results, first vs second run, comparison with the paper, health check. Part C (16-22): research use, benefits, faculty slides, script, Q&A, next steps, GitHub.

PART A · UNDERSTAND THE PAPER

1. What problem does the paper study?

An automatic license plate recognition (ALPR) system has two steps: **detection** (find the plate in the photo) and **recognition** (read its characters, an OCR task). The paper looks only at recognition, because modern systems crop the plate and **rectify** it (warp it to a flat rectangle) before reading it (p.1).

Researchers compare models on public datasets that are divided into train and test sets. The split is usually by *file*: each image goes to train or to test. But one vehicle is often photographed several times (another second, another angle, another subset). After cropping and rectifying, two such photos of the same plate look almost identical. If one is in train and the other is in test, the model is partly tested on something it has already memorised (p.1, Fig. 1-3).

So the question is: **how much of the reported accuracy is memory and how much is generalisation?** The idea comes from Barz & Denzler (2020), who found the same problem in CIFAR-10/100 ("Do we train on test data? Purging CIFAR of near-duplicates") (p.2).

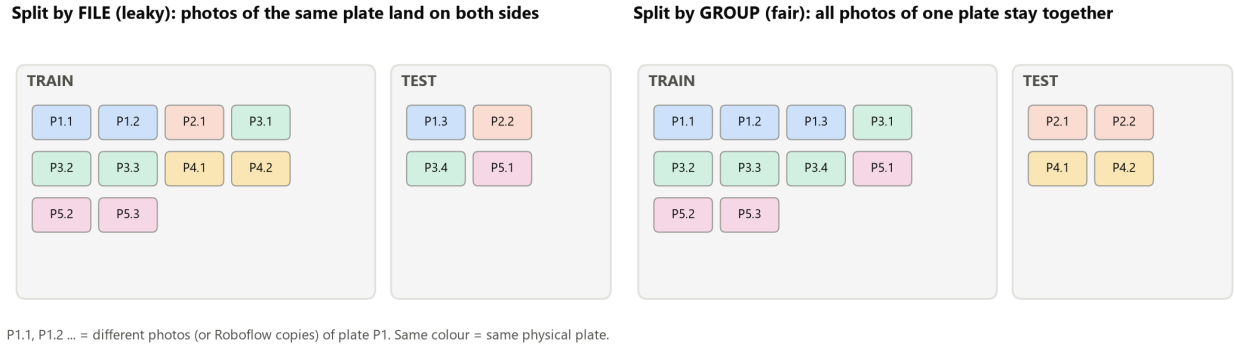


Figure 1 (drawn for this report). Left: a split by file lets photos of one plate land in both train and test. Right: a split by group keeps every photo of a plate on one side. This is the whole idea of the paper.

2. Key terms in plain words

Term	Meaning	Example
Exact duplicate	the same file twice (byte copy) or a re-encoded copy	the same JPEG saved twice
Near-duplicate (this paper)	**different images that show the same license plate** (p.1)	one car photographed in subset AC and again in subset RP
Roboflow copy (our data)	an augmented copy of one photo (noise, brightness, flip) exported as a new file	KUET: about 3 files per photo in train
Data leakage	information from the test set is present during training	test plate also in train
Fair split / duplicate-free split	a split where no test image has a near-duplicate in train	AOLP-Fair-A, CCPD-Fair
Group split	put all images of one group (plate, vehicle, photo cluster) into the same split	our 70/15/15 split, seed 42
Recognition rate	plates read 100 % correctly / all test plates (one wrong character = wrong)	96.94 %
Gap	original accuracy minus fair accuracy, in percentage points (pp)	99.18 - 96.94 = 2.24 pp
Relative gap	gap / original error rate = gap / (100 % - original accuracy) (p.4)	2.24 / 0.82 = 273 %
Memorisation vs generalisation	remembering seen examples vs working on new ones	what a test set must measure

> **Why the relative gap matters.** A drop from 99.18 % to 96.94 % sounds small (2.24 points), but the error goes from 0.82 % to 3.06 %, which is 3.7 times as many wrong plates. The relative gap shows this: 2.24 / 0.82 = 273 %. When accuracy is near 100 %, always look at the error rate.

3. Datasets and how people split them

Dataset	What it is	Split used in the literature	Test images with a near-duplicate in train (p.3, p.5)
AOLP (Taiwan)	2,049 images in 3 subsets: access control AC 681, law enforcement LE 757, road patrol RP 611 (p.2)	**AOLP-A**: random 2:1 train/test. **AOLP-B**: train on AC+LE, test on RP. 20 % of train is validation.	AOLP-A: 320 of 683 = **46.9 %** AOLP-B: 413 of 611 = **67.6 %**
CCPD (mainland China)	366,789 images taken with handheld cameras in one city; subsets Base, Blur, Challenge, DB, FN, Green, Rotate, Tilt, Weather (p.2)	standard split: Base 200k into 100k train + 100k val; all other subsets (about 157k) are test	29,943 of about 157k = **19.1 %**
ReId	182,335 low-resolution plate crops, standard split 105,923 / 76,412 (p.5)	standard split	52,394 of 76,412 = **68.6 %**

Note that AOLP-B, the "harder" protocol that tests on a different subset, has the *most* duplicates: the same cars drive through the access-control gate and past the road-patrol camera.

4. How the authors found the near-duplicates

1. **Step 1, filter by plate text.** Only images whose plates have the same text can be duplicates (p.3).
2. **Step 2, compare pixels.** Among those, they crop and rectify the plates and take the nearest neighbour by Euclidean distance in pixel space. Because step 1 already removed different plates, this simple distance "works reasonably well" (p.3).
3. **Step 3, check by eye.** Fig. 1 shows pairs at the 10th, 50th and 90th percentile of distance. Even the 90th-percentile pair is clearly the same plate.

Their lists of duplicates and the fair splits are public at `raysonlaroca.github.io/supp/lpr-train-on-test/`. There is no training code.

5. How they built the fair splits

They could not replace duplicates with new images (that would change the data distribution), so they re-split while keeping each protocol's character and size (p.3-4):

Fair split	What they changed	What stayed the same	Why this design
AOLP-Fair-A	new random 2:1 split, but all images of one vehicle/plate are kept in the same set	same numbers of train, val and test images	a like-for-like random split, only without leakage
AOLP-Fair-B	**only removed** the duplicates from the test set (RP)	train and val identical to AOLP-B	so nobody can say the drop comes from less training data
CCPD-Fair	Base is still split 100k/100k, but every duplicate goes to **validation**, not train (same for Green)	test set unchanged (the same 157k images)	the test is identical, so any drop is caused by the training data

> **The clever part.** Fair-B and CCPD-Fair each keep one side fixed (train or test). That removes the obvious objection "your fair split is just harder or smaller". Our own experiment copies these two ideas: the "clean part of the published test" is the Fair-B idea, and the seen/unseen check is the controlled version.

6. Models and training

Item	Detail (p.4)
Models (6)	LPR-specific multi-task CNNs: CNNG, Holistic-CNN, Multi-Task. Scene-text models: STAR-Net (CTC), TRBA (attention), ViTSTR-Base (Transformer)
Keras models	Adam, learning rate 1e-3 (ReduceLROnPlateau patience 3, factor 0.1), batch 64, at most 100 epochs, early stop patience 7
PyTorch models	deep-text-recognition-benchmark fork; Adadelta (rho 0.99), 300k iterations, batch 128
Extra data	many synthetic plates: blank templates + random but realistic character sequences + perspective, noise, shadow and colour changes (also applied to real images) (p.4, Fig. 4)
Hardware	AMD Threadripper 1920X, 96 GB RAM, NVIDIA Quadro RTX 8000 (48 GB). Training time not reported

Fairness of comparison	all models trained by the authors with the same protocol in each experiment; no numbers copied from other papers (p.4, footnote 6)

7. Results of the paper

Recognition rate (%) on the original split → the fair split (gap in points; relative gap). Best in each column in bold. Source: Tables I, II, III (p.4-5).

Model	AOLP-A → AOLP-Fair-A	AOLP-B → AOLP-Fair-B	CCPD → CCPD-Fair
CNNG	98.88 → 95.63 (3.25; 290 %)	98.91 → 96.80 (2.11; 194 %)	88.24 → 86.93 (1.31; 11 %)
Holistic-CNN	96.75 → 93.11 (3.64; 112 %)	98.42 → 96.30 (2.12; 134 %)	77.01 → 75.41 (1.60; 7 %)
Multi-Task	97.33 → 93.79 (3.54; 133 %)	98.42 → 95.29 (3.13; 198 %)	83.01 → 81.84 (1.17; 7 %)
STAR-Net	98.69 → 95.83 (2.86; 218 %)	98.47 → 96.46 (2.01; 131 %)	78.53 → 73.33 (**5.20**; 24 %)
TRBA	**99.18** → **96.94** (2.24; 273 %)	98.75 → **97.47** (1.28; 102 %)	75.83 → 71.48 (4.35; 18 %)
ViTSTR-Base	98.74 → **96.94** (1.80; 143 %)	98.75 → 97.31 (1.44; 115 %)	79.06 → 76.37 (2.69; 13 %)

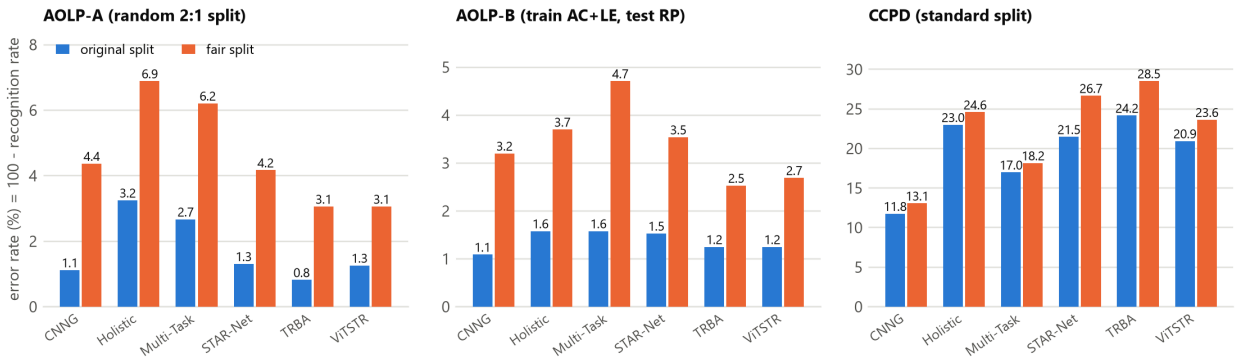


Figure 2 (re-plotted from Tables I-III). Error rate = 100 % - recognition rate. On AOLP the orange bars (fair split) are 2-4 times as high as the blue ones (original split).

The four findings to remember

- **Errors more than double on AOLP.** Every model, both protocols. Relative gaps 102-290 % (p.4).
- **The ranking changes.** On AOLP-B, CNNG is 1st on the original split but only 3rd on the fair split. On AOLP-A, ViTSTR moves from 3rd to tied 1st (p.4-5). So a "new best model" claimed on a leaky split may only be better at memorising.
- **CCPD: smaller relative effect, big absolute effect.** Average 80.3 % → 77.6 %. STAR-Net loses 5.20 points and TRBA 4.35. Even the smallest gap (1.17 pp) is 1,800+ extra misread plates, and 5.2 pp is 8,000+ (p.5). The worst subset is Blur: TRBA 50.2 % → 38.7 % (Table IV).
- **Bigger models may hide it longer.** The ability to memorise grows with model capacity, so the problem may get worse as models get deeper (p.5).

Section IV: the problem is everywhere

- EnglishLP, Medialab LPR and PKU have no official split; people split randomly and get near-duplicates without noticing (p.5, Fig. 5).
- ReId: 68.6 % of test images have a near-duplicate in train, and works using it never mention it (p.5).
- **Across datasets:** CLPD (used as a "cross-dataset" test set) shares vehicles with ChineseLP. Training on one and testing on the other is then not really cross-dataset (p.6, Fig. 7).
- Good example: RodoSol-ALPR was split with no duplicates by design (p.6).

8. Critical reading: strengths and limits

Strengths

- Simple, important question; clear evidence on the two most used datasets and six model families.
- Careful control in AOLP-Fair-B and CCPD-Fair (one side kept fixed), so the drop cannot be blamed on less data.

- All models trained by the authors with one protocol, so the comparison is consistent.
- Duplicate lists and fair splits are public, so anyone can check or reuse them.
- Peer-reviewed (IJCNN 2023, IEEE), short and easy to cite.

Limits (good points to raise yourself before the faculty does)

- **Recognition only.** Detection is not tested. The authors argue full images vary more, but they give no numbers. (Our run adds detection.)
- **Duplicates need plate text.** Step 1 uses the plate text, so unlabelled images and duplicates of *different* plates of the same car are not covered.
- **AOLP-Fair-A changes which images are trained on**, so part of its drop may come from a different (maybe harder) test set. Fair-B controls for this, and Fair-B still shows big gaps.
- **One run per model.** No seeds, no confidence intervals. Small gaps (for example 1.28 pp on 611 images) could partly be noise.
- **No training code**, only the splits. CCPD corner labels are imprecise (p.2, footnote 2), which affects rectification.
- Synthetic data was added for all models. It reduces memorisation, so without it the gaps might be even larger. This is not tested.

9. Check yourself (2 minutes, without the paper)

> In AOLP and CCPD, many test plates also appear (as other photos) in training: 47-68 % of AOLP test images and 19 % of CCPD's. Re-splitting so that no plate is on both sides makes six OCR models' error rates more than double on AOLP and drops CCPD's average from 80.3 % to 77.6 %. The "best" model even changes. Lesson: split by plate or photo group, never by file, or the numbers are inflated.
>
> Two weak points: OCR only (no detection), and duplicates are found through plate text (unlabelled images are not covered). What we take from it: our grouped split, our leakage audit, and the leaky-vs-fair experiment on Bangla data.

10. What we ran (dataset v1.1)

We applied the paper's idea to **KUET** (Bangladeshi License Plate Recognition Dataset, Hossain & Hassan; the dataset of paper #1 in our list). KUET has a plate-localisation part and a character-recognition part, each with its own published train/valid/test split exported from Roboflow. The same source is trained in two versions, with the same models:

Version	Data	Detection images train / val / test	Reading crops train / val / test
leaky	KUET's own published split with every Roboflow copy (`extras/leaky_kuet/`). This is how papers that use KUET trained.	5,620 / 277 / 553	3,391 / 320 / 194
fair	the same KUET images in our grouped split, seed 42, one copy per photo (`fair_kuet_*.txt`). Reading test = **the same 194 published test crops**; the 315 other crops of those plates are left out of training.	1,758 / 311 / 540	660 / 122 / 194

Setting	Value
Plate detector	YOLOv8n (3.01 M parameters, 8.2 GFLOPs) from COCO weights; 640 px; batch 16. Leaky 50 epochs, fair 160 epochs: both about 17,600 optimiser steps
Plate reader	YOLOv8n character detector over 101 tokens (10 digits, the word "Metro", 26 series letters, 64 district words); 320 px; batch 32. A "rows" decoder turns the boxes into the two-row BRTA plate text. Leaky 80 epochs (8,480 steps). Fair: 404 epochs planned for equal steps, stopped early at 377 (best epoch 277, 7,917 steps)
Training	SGD, lr0 0.01, seed 42, deterministic; Ultralytics 8.4.170, PyTorch 2.11.0+cu128, Python 3.13.15; dataset_version 1.1
Hardware / time	Kaggle 2× Tesla T4. GPU 0: det_leaky 50.3 min → det_fair 43.2 min → scoring 1.2 min. GPU 1: rec_leaky 29.5 min → rec_fair 29.3 min. Wall time 96.3 min
Metrics	detection: mAP50 and mAP50-95; reading: plate exact % (the paper's recognition rate: whole plate correct) + exact McNemar test on the same plates
Test sets	**published test** (what papers report; for reading now also the fair test); its **clean** part (no near-duplicate in the published train) and **duplicated** part; the fair detection test (540 photos)

> **This is the paper's CCPD-Fair design.** In CCPD-Fair the test set is unchanged and only the training set loses the duplicates of test images (p.3-4). Our v1.1 reading experiment does the same: one test set (194 plates) and two training sets, one with and one without the test plates. Any difference is caused by what was in training.

What counts as a duplicate. Detection: a test photo in the same near-duplicate cluster as a train photo (perceptual hashes checked by cross-correlation, Roboflow base names, MD5). Reading: a test crop whose **plate text** occurs in train. No KUET reading test crop is the same *photo* as a train crop, so all of these are "same plate, other photo": the paper's definition.

11. How leaky is KUET's published split?

Measure	KUET detection	KUET reading	Paper, for scale
published train vs distinct content	5,687 files from only 1,964 distinct photos (about 2.9 copies each); 5,620 used after label checks	3,391 crops of only 518 plates (about 6.5 crops per plate)	-
test items with a near-duplicate in train	79 of 553 = **14.3 %**	114 of 194 = **58.8 %**	AOLP-A 46.9 %, AOLP-B 67.6 %, CCPD 19.1 %
our fair training set	1,758 photos, none of the test photos	660 crops of 378 plates, **none** of the 194 test plates	-

KUET's reading test set is as leaky as AOLP: more than half of its test plates were already seen in training. The same audit (`dataset/docs/leakage_audit.md`) finds that BLPD (paper #2) is worse for detection: 94 of 213 test photos (44.1 %) have a copy in train, and 270 of 721 reading crops (37.4 %).

12. Results of the v1.1 run

12.1 Plate reading: the same 194 plates, two training sets

Leaky reader (trained with the test plates)

85.57 %

95 % interval 79.9-89.8

Fair reader (test plates kept out)

77.84 %

95 % interval 71.5-83.1

The gap is **7.73 points**. The error rate grows from 14.43 % to 22.16 % (×1.54). Relative gap (paper formula) = 7.73 / 14.43 = **54 %**. Because both readers read the same plates, a paired test is possible: 144 plates both read right, 22 only the leaky reader, 7 only the fair reader, and 21 neither. The exact McNemar test gives **p = 0.008**, so the difference is not luck.

Which plates	plates	leaky %	fair %	difference	only leaky right / only fair right	McNemar p
all KUET test plates	194	85.57	77.84	**+7.73**	22 / 7	**0.008**
B: the plate is in the leaky training set (other photos)	114	90.35	79.82	**+10.53**	17 / 5	**0.017**
C: the plate is in neither training set	80	78.75	75.00	+3.75	5 / 2	0.45
other datasets (anpr, blpd, lpdba, nightlpbd), unseen by both	468	50.21	50.00	+0.21	26 / 25	1.0

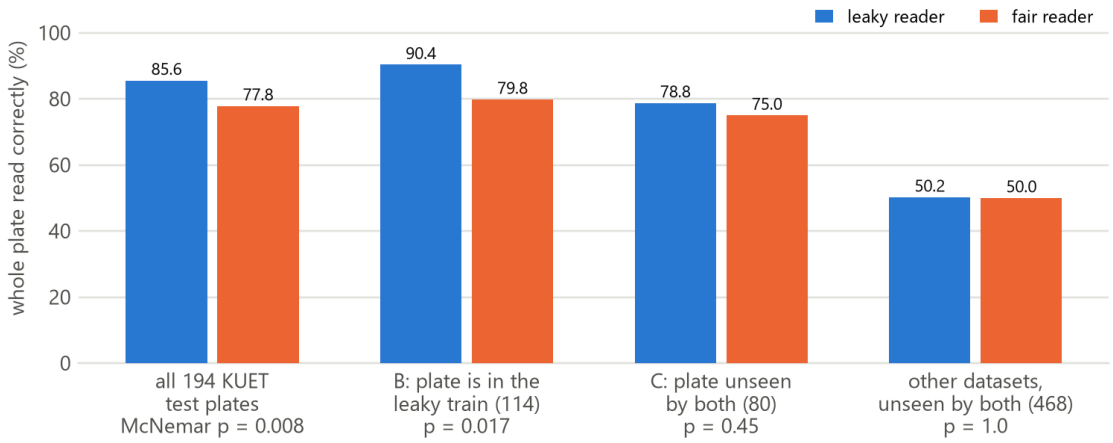


Figure 3. The same plates, read by the leaky and the fair reader. The leaky reader is clearly better only on the plates it had seen in training (B). On new KUET plates (C) and on other datasets the difference is small and not significant.

> **This is the strongest result of the study.** The paper says leaky splits measure memory. Here we see it directly: the same test plates, the same metric, and a paired test. The leaky reader's lead is significant only where it had seen the plate (group B: the paper's "same plate, other photo"). On new plates its extra training copies do not make it significantly better.

The same-model check (the paper's AOLP-Fair-B idea: one model, duplicates removed from the test) gives the same picture: the leaky reader reads 85.57 % of all published test plates but 78.75 % of its clean part (6.82 points; relative gap 47 %). Group C has 80 plates, so its +3.75 might be a small real effect of the larger training set, but the test cannot tell (p = 0.45). By condition on the other datasets: night (59 plates) 50.85 % vs 59.32 % (p = 0.23); blurry (31) 74.19 % vs 64.52 % (p = 0.25); clear (378) 48.15 % vs 47.35 % (p = 0.74). None of these differences is significant.

12.2 Plate detection: memory shows on the duplicated photos

Detector (mAP50-95 %)	published test, all (553)	clean part (474)	duplicated part (79)	fair test (540)
det_leaky	76.86	76.04	**83.06**	77.45* (76.78 on the 520 it never saw)
det_fair, v1.1 (160 epochs, same steps)	71.99	73.06	68.51	73.31
difference	+4.87	+2.98	**+14.55**	+3.47 (unseen photos)
det_fair, first run (50 epochs)	74.45	75.93	68.24	74.68

* det_leaky trained on 20 of the 540 fair-test photos (identical files). det_fair trained on none of KUET's published test photos. mAP50 is 98.5-99.5 % in every cell, so only the stricter mAP50-95 shows differences.

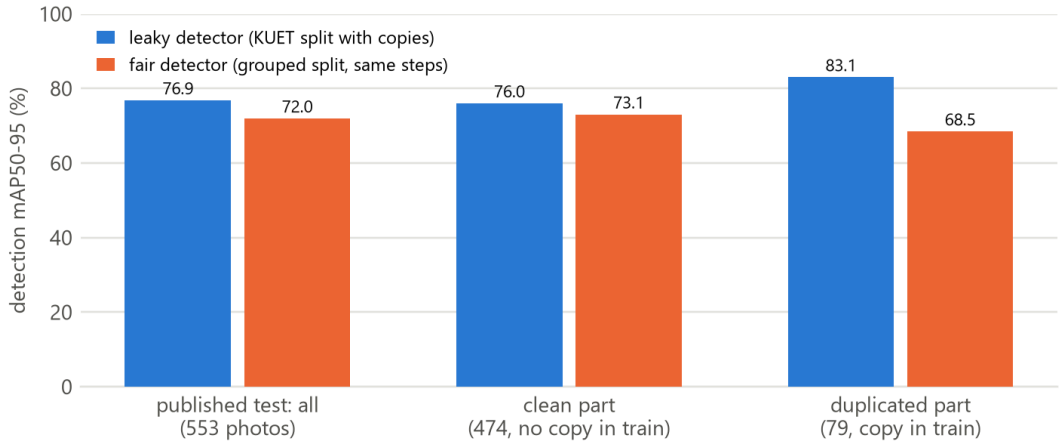


Figure 4. Detection on KUET's published test. On clean photos the leaky detector is about 3 points better. On the 79 photos that have a copy in its training set it is 14.6 points better.

How to read it. On clean photos the leaky detector is about 3 points better. It saw 1,964 distinct photos plus their Roboflow copies, against 1,758 photos for the fair detector, so it had more data and more augmentation. On the 79 photos with a copy in its training set it is 14.6 points better. Compared with its own clean score, the leaky detector *gains* 7.0 points on the duplicated photos, while the fair detector *loses* 4.6 points there because they are harder photos. That 11.6-point swing is memory (14.7 in the first run). At IoU 0.5 (mAP50) plate detection is solved on KUET, so leakage does not show in the headline number.

> **Longer training did not help the fair detector.** With 160 epochs its best epoch was 102 and its validation box loss rose after about epoch 110. Its clean-photo score (73.06) is lower than the 50-epoch first run (75.93). So "the same number of steps" is not automatically fairer for a small training set. The clean-photo difference (0.1 or 3.0 points) depends on this choice. The duplicated-photo advantage (14.5-14.8 points) does not.

13. First run (v1.0) vs this run (v1.1): which numbers to use

	First run, 18 Oct (dataset v1.0)	This run, 22 Oct (dataset v1.1)
fair reading test	183 KUET plates of our main split: **different** plates from the leaky test	**the same** 194 published test plates
fair reading train	908 crops, including 135 of the published test crops	660 crops, none of the test plates
fair training length	det 50 epochs, reader 80 epochs (about 3× fewer steps)	det 160 epochs, reader 377 epochs (about the same steps)
leaky reader / leaky detector	85.57 % / 76.86 mAP50-95	identical: 85.57 % / 76.86 (deterministic training)
fair reader	72.13 % (on its own 183 plates)	77.84 % (on the same 194 plates)
reading gap	13.44 points (×1.93 error), two different test sets	**7.73 points (×1.54 error), same test, p = 0.008**

seen / unseen check	from saved predictions; unseen group 38 plates: -2.63	built into the kit; unseen by both 80 plates: +3.75 (p = 0.45)
fair detector, clean photos	75.93	73.06 (overfits with longer training)
run time	65.5 min	96.3 min

> **Use the v1.1 numbers.** The first run compared the readers on two different sets of test plates, and its fair reader had trained on many published test crops. Dataset v1.1 fixed both problems. The direction of the result did not change (leakage inflates reading accuracy), but the honest size is 7.7 points, not 13.4. That we found and corrected this ourselves is worth telling the faculty: it shows the evaluation was checked.

14. Our results next to the paper

	Paper (AOLP / CCPD)	Ours (KUET, v1.1)
test items with a near-duplicate in train	AOLP-A 46.9 %, AOLP-B 67.6 %, CCPD 19.1 % (p.3)	reading 58.8 %; detection 14.3 %
same test, duplicates removed from train (CCPD-Fair)	CCPD average 80.3 → 77.6 % (-2.7); up to -5.20 (Table III)	85.57 → 77.84 % (**-7.73**)
relative gap of that (gap / original error)	CCPD 7-24 %	54 %
same model, duplicates removed from test (AOLP-Fair-B)	-1.28 to -3.13 points (102-198 %)	-6.82 points (47 %)
random re-split by group (AOLP-Fair-A)	-1.80 to -3.64 points (112-290 %)	not done
significance test	none reported	McNemar p = 0.008 (all), 0.017 (seen plates), 0.45 (unseen)
model ranking changes?	yes on AOLP	not tested (one reader architecture)
detection	not studied	duplicated photos +14.6 mAP50-95, clean +3.0
runs / seeds	one run per model	one seed; the leaky runs reproduced exactly in two runs

Our absolute gap is larger than any in the paper. KUET's reading split repeats each plate about 6.5 times in training, and 59 % of the test plates are among them (CCPD: 19 %). The relative gaps lie between CCPD's and AOLP's because our reader starts at 14 % error: AOLP models start near 1 %, where a few points of drop is a large multiple.

> **Do not compare 85.57 % with the paper's 99.18 %.** Different country, plate format, data and models. Only compare the *direction and size of the gap*.

15. Is my training OK? Health check of the v1.1 run

Yes, the run is correct and complete.

- All five jobs ended with `code=0`: det_leaky 50.3 min, rec_leaky 29.5 min, rec_fair 29.3 min, det_fair 43.2 min, leaky_vs_fair 1.2 min.
- `environment.json` says `dataset_version: 1.1`, and the fair reading test is the 194 published test crops, as planned.
- Epochs: 50/50, 160/160 and 80/80. rec_fair stopped at **377 of 404** because Ultralytics early stopping saw no validation gain for 100 epochs (best epoch 277). This is normal, not an error.
- Every dataset scan reports **0 corrupt** images; no NaN losses.
- **Reproducible:** both leaky models give exactly the same numbers as on 18 October (85.57 %, 76.86 mAP50-95 and every subset). I also re-scored both readers on this PC: 85.57 % and 77.84 % again.
- Both zips were packed: `paper23_outputs.zip` 51.3 MB (with weights) and `paper23_results.zip` 3.5 MB.

Things to know before you quote numbers (not crashes)

#	Issue	Effect	What to do
1	**The fair detector overfits with 160 epochs** (best epoch 102, validation box loss rising after about 110).	Its clean-photo score is 2.9 points below the 50-epoch first run, so the clean difference (+3.0) is partly the recipe.	Quote the duplicated-photo result (+14.6) as the main detection finding; mention both fair runs.
2	**The per-condition reading table that includes KUET crops** (rec_fair 62.37 % vs rec_leaky 60.68 %).	Both readers trained on part of the 183 KUET crops of the main OCR test (a different split).	Use the "without KUET" table (50.21 % vs 50.00 %), which the kit prints too.
3	**The seen/unseen table prints every row twice** (leaky test and fair test are now the same 194 crops).	Cosmetic.	Read one half.
4	**Row label "each on its own test"** in results_vs_paper.md.	Out of date: in v1.1 both readers use the same test.	Read it as "on the same 194 plates".
5	**One dataset, one seed.**	95 % intervals are about ±5 points (±9 on group C).	McNemar covers the reading result; add seeds and BLPD later.
6	**xsort decoder = 0 %** and whole OCR test 50-62 %.	Expected: two-row plates; both readers trained on KUET only.	Use the rows decoder; read the other sources as cross-dataset.
7	**The first report's headline** (85.57 vs 72.13, ×1.93, group C -2.6).	Superseded (section 13).	Use only the v1.1 numbers.

16. How to use this paper in your research

Use	What to do concretely
1. Justify our dataset design	In the report's Method section, cite Laroca et al. (2023) and our numbers to explain why the BanglaLPR All-in-One split is grouped (near-duplicate clusters + plate text + crops of one photo = one group). Sample sentence: *"Following Laroca et al. [x], who showed that near-duplicates between training and test sets inflate LPR accuracy, we split by group. On KUET's published split, 58.8 % of the test plates also occur in training; a reader trained on that split reads 85.6 % of the test plates, but only 77.8 % when those plates are kept out of training (McNemar p = 0.008), and it is no better on unseen plates."*
2. Read published Bangla numbers carefully	Papers that train and test on KUET's or BLPD's own split (for example #1 and #2 in our list) report numbers that include this leakage. When we compare with them, write "on the published (leaky) split" and give our fair number next to it.
3. A reporting rule for the team	Never re-split the dataset randomly. Report on our fair test. For any plate reader also report **seen vs unseen plates**. When two models read the same test plates, use a **paired test** (McNemar), as in section 12.1.
4. Make the leakage audit a result	`dataset/docs/leakage_audit.md` covers 14 public Bangla splits (KUET 14.3 % / 58.8 %, BLPD 44.1 % / 37.4 %, NightLPBD 20.7 % / 27.6 %, Zenodo 22.1 %). Before calling it new, search Google Scholar for a similar audit of Bangla datasets. If you find none, it can be a section of the thesis or a short paper: "near-duplicates in Bangladeshi LPR datasets".
5. Choose models fairly	The paper shows the ranking can change. Compare all our models (YOLO char reader, CRNN, PARSeq, TrOCR...) only on the grouped test, or a "better" model may just memorise better.
6. Check across datasets too	Section IV warns about duplicates across datasets. Our build de-duplicated all 12 sources together, so a plate copied between two public sets is one group. Our "other datasets" test (468 plates) is a true cross-dataset test.

17. What you gain from it

- **Credible numbers.** Faculty and reviewers can trust that our accuracy is not memory.
- **A defence, not a weakness.** When someone asks "why is your accuracy lower than paper X?", you have a peer-reviewed reason and our own measurement.
- **Our own finding on Bangla data,** not only a reproduction: 85.6 → 77.8 % on the same plates, significant, explained by seen plates.
- **A careful method you can show:** same test set, paired test, seen/unseen groups, a correction between two runs, and an exact reproduction.
- **Fair model choice.** Decisions based on the grouped test pick the model that generalises.
- **A reusable tool and rule** for the whole team (grouped split, leakage audit, seen/unseen breakdown, McNemar).

18. What to show the faculty (8 slides)

#	Slide	Content (all files are in this folder or the dataset)
1	The claim	"Public Bangla LPR splits leak test plates into training; on KUET this makes plate reading look 7.7 points better than it is (p = 0.008)."
2	The problem	Figure 1 of this report (split by file vs by group).
3	The paper	Laroca et al., IJCNN 2023: duplicate fractions (46.9 / 67.6 / 19.1 %) + Figure 2 (errors double).
4	Our audit	Table from leakage_audit.md: KUET, BLPD, NightLPBD, Zenodo duplicate rates.
5	Our experiment	One test set (194 KUET plates), two training sets (with / without the test plates), same model and steps (section 10).
6	Headline result	85.57 % → 77.84 %; error ×1.54; McNemar p = 0.008 (section 12.1).
7	Proof it is memory	Figure 3 (seen +10.5, p = 0.017; unseen +3.75, p = 0.45; other datasets 50.2 vs 50.0) and Figure 4 (detection: duplicated photos +14.6).
8	So what + next	Our grouped split; our numbers are honest; we corrected our first run (section 13); limits (one dataset and seed); next steps (section 21).

19. What to explain: a 2-minute script

> "Paper #23 is by Laroca and colleagues, IJCNN 2023. They noticed that the most used license-plate datasets put different photos of the *same plate* in both train and test. After cropping, these photos look almost the same, so the model is partly tested on what it memorised. In AOLP half to two-thirds of the test images have such a near-duplicate. When they removed them, error rates of six OCR models more than doubled, and the best model changed.
>
> We checked our Bangla data the same way. In KUET's published split, 59 % of the test plates also appear in training, because Roboflow exports several copies of each photo and the same cars appear many times. We trained the same YOLOv8n reader twice, once with and once without those plates, and tested both on the same 194 plates. The first reads 85.6 % and the second 77.8 %; a paired test gives p = 0.008. The whole difference is on plates the first model had seen. On new plates and on other datasets the two models are equal.
>
> That is why our dataset is split by group, and why our numbers can be lower than some published Bangla results: ours measure generalisation."

20. Questions the faculty may ask, with answers

Q1. What exactly is a near-duplicate?
Different images showing the same physical plate (the paper's definition). In our data also Roboflow-augmented copies of one photo. We find them with perceptual hashes checked by cross-correlation, Roboflow base names, MD5, and the plate text.

Q2. Is the fair reader worse just because it has less training data?
We checked this. It trained on 660 crops of 378 plates; the leaky one on 3,391 crops of 518 plates, 114 of them test plates. On plates neither saw it is 3.75 points behind, which is not significant (p = 0.45), and on 468 plates from other datasets the two are equal (50.2 vs 50.0 %). The significant gap (+10.5, p = 0.017) is on the plates the leaky reader had seen. So most of the 7.7 points is memory.

Q3. Is the difference significant? You have one run.
Both readers read the same 194 plates, so we used the exact McNemar test: 22 plates only the leaky reader got right, 7 only the fair one, p = 0.008. The leaky models gave identical numbers in two separate runs (deterministic training). More seeds are planned.

Q4. Does it matter for detection?
Less for the headline. mAP50 is about 99 % either way. At strict IoU (mAP50-95), on the 79 duplicated photos the leaky detector is 14.6 points better, while on clean photos it is 3 points better, so most of its advantage on duplicated photos is memory.

Q5. Why not just delete duplicates from the test set?
We did both. One model with duplicates removed from the test (the paper's Fair-B idea): 85.57 → 78.75 %. One test set, duplicates removed from training (the paper's CCPD-Fair idea): 85.57 → 77.84 %. Both give the same direction and a similar size.

Q6. Why is your accuracy lower than published Bangla papers?
Many published results use the leaky published splits. Our test is grouped and covers five sources. On KUET alone, leakage is worth 7.7 points of plate reading.

Q7. Your first report said 13.4 points. Why did it change?
In the first run the fair reader was tested on a different set of KUET plates and had trained on many published test crops. We found this, fixed the lists in dataset v1.1, and re-ran. Now both readers use the same 194 plates. The corrected gap is 7.7 points, in the same direction.

Q8. Doesn't removing copies throw away useful augmentation?
We remove only duplicate *files*. Augmentation is done online during training (YOLO mosaic, HSV, flips), so the model still sees variations. It just does not see them from the test set.

Q9. What would you change next?
3 seeds; repeat on BLPD (44 % leaky detection test); a stronger reader (PARSeq, paper #31); early stopping for the fair detector.

21. Next steps (in order)

1. **Upload the results** (section 22). Fill the team table: put `paper23_results.zip` in `docs/results/` and run `python add_results.py paper23_results.zip --member "Rokib Hasan Oli"` (it labels the run v1.1).
2. **Write it up:** one paragraph + Figure 3 in the Method/Dataset chapter; the audit table in the Dataset chapter.
3. **3 seeds** of the reader pair; report mean ± std next to McNemar.
4. **Repeat on BLPD** (44.1 % of detection test photos leaky), the dataset of paper #2.
5. **Stronger reader:** leaky vs fair with PARSeq (kit #31). The paper expects higher-capacity models to memorise more.
6. **Small kit clean-ups:** print the seen/unseen table once when the two tests are equal; update the "each on its own test" label; add early stopping (patience) to the fair detector.

22. Uploading to GitHub (and Drive)

Everything is prepared in `After Train/github_upload/weights/paper23/MD.Rokib.Hasan.Oli/`, in the same layout as Md. Yousuf's `weights/paper03/Md.Yousuf/` (two-digit paper number):

File	Size	Why
README.md	-	what the package is, main results, limits
best_det_leaky.pt, best_det_fair.pt	6.0 MiB each	the two plate detectors
best_rec_leaky.pt, best_rec_fair.pt	6.6 MiB each	the two plate readers
paper23_results.zip	3.3 MiB	all results, logs, curves and code, without weights
results_vs_paper.md	-	the kit's ours-vs-paper table
paired_test_v11.json	-	the McNemar test of section 12.1
Paper23_Study_Report.pdf	-	this report

Not to GitHub: `paper23_outputs.zip` (48.9 MiB, includes last.pt) goes to the team Drive folder `498R / weights / paper23 /`. It is ready in `After Train/for_drive/`. Never upload `results.zip` (174 MB, the whole Kaggle output).

Way 1: in the browser (easiest)

1. Open github.com/Rokib-Hasan-Oli/CSE498R and click the `weights` folder.
2. Click **Add file → Upload files**.
3. From File Explorer, drag the whole folder `After Train\github_upload\weights\paper23` into the page (Chrome/Edge keep the folder structure). Check that the list shows `paper23/MD.Rokib.Hasan.Oli/...` with 9 files.
4. Commit message: `Add paper #23 results on dataset v1.1: leaky vs fair KUET split (Rokib).`
5. Choose **Commit directly to the main branch → Commit changes**. Open the folder on GitHub and check that the README is shown.

Way 2: with git (Git is installed on this PC)

```
cd D:\498R
git clone https://github.com/Rokib-Hasan-Oli/CSE498R.git
   (now copy the folder After Train\github_upload\weights\paper23
    into D:\498R\CSE498R\weights\ with File Explorer)
cd CSE498R
git add weights/paper23
git commit -m "Add paper #23 results on dataset v1.1 (Rokib)"
git push
```

The first push asks you to sign in to GitHub in the browser. Files under 100 MB are fine with git (the web upload limit is 25 MB per file). The repo tracks `*.zip` with Git LFS; with git the zip goes to LFS automatically, while a web upload stores it as a plain file (as happened with earlier uploads).

Appendix: where every number comes from

Number	Source file
paper numbers	arXiv 2304.04653v2, pages given in the text (Tables I-IV on p.4-5)

v1.1 scores, gaps, seen/unseen, training length	`results.zip → runs/leaky_vs_fair/results.json, report_extra.md`
McNemar tests, 95 % intervals	`paired_test_v11.json` (`report_src/paired_v11.py`: both best.pt re-read the 194 crops on this PC; the 468 other plates come from `runs/rec_*/ocr_preds_test.csv`)
training settings, times, curves	`runs/<run>/results.json, args.yaml, results.csv`; Kaggle log `paper-23.log`
first-run numbers	the report of 18 October (`Paper23_Study_Report_v1.0_run.pdf`)
KUET/BLPD leakage, copies per photo	`dataset/docs/leakage_audit.md`, `dedupe_report.md`

```
@inproceedings{laroca2023do,
  title     = {Do We Train on Test Data? {T}he Impact of Near-Duplicates on License Plate Recognition},
  author    = {Laroca, Rayson and Estevam, Valter and Britto Jr., Alceu S.
               and Minetto, Rodrigo and Menotti, David},
  booktitle = {International Joint Conference on Neural Networks (IJCNN)},
  pages     = {1--8},
  year      = {2023},
  doi       = {10.1109/IJCNN54540.2023.10191584}
}
```